

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Mini-Project — Phase 1

Software Requirements Specification

for

TaskFlow — Collaborative Task Manager Application

Version 1.0 – Draft for Review

Prepared by: Ankita Pramod, Ashritha Kamalakar Hegde, B Nuthan, Chaithanya B (Team 2)

PES University, Bangalore — UE24CS341A Software Engineering

05 September 2026

Item	Entry
Project ID	TM-2026-02
Group Number	Team 2
Course	UE24CS341A — Software Engineering
Technology	JavaScript — React.js (frontend) + Node.js/Express (backend), PostgreSQL
Development Methodology	Agile (Scrum, 6 sprints)
Related Documents	TM-2026-02_Synopsis.pdf (Deliverable 1)

Table of Contents

Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Intended Audience and Reading Suggestions 1
1.3 Product Scope 1
1.4 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 3
2.6 Assumptions and Dependencies 3
3. External Interface Requirements 3
4. Analysis Models 4
5. System Features (F1–F8) 6
6. Other Nonfunctional Requirements 14
7. Other Requirements 16
Appendix A: Glossary 16
Appendix B: Field Layouts 17
Appendix C: Requirement Traceability Matrix 18
Appendix D: Initial Validation Specification 19
Appendix E: Project Plan 20

Revision History

Name	Date	Reason For Changes	Version
Team 2	02 Sep 2026	Initial synopsis approved	0.1
Team 2	05 Sep 2026	First complete SRS draft for team review	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of TaskFlow, a collaborative task manager application for small teams, developed by Team 2 (Project ID TM-2026-02) for the UE24CS341A Software Engineering mini-project. It covers the complete first release: user accounts, task management, projects, notifications, dashboards and administration (features F1–F8, as introduced in the approved synopsis).

1.2 Intended Audience and Reading Suggestions

This SRS is written for: the four development team members, who use it as the single source of truth for what to build and test; the course faculty/evaluators, who use it to assess requirement quality and traceability; and any future maintainer of the codebase. Section 2 gives a manager-level overview; Section 3 and 4 describe interfaces and models; Section 5 is the detailed, developer-facing requirement list organized by feature; Section 6 covers non-functional requirements; the appendices give supporting reference material (glossary, field layouts, traceability, validation plan, and project plan).

1.3 Product Scope

TaskFlow lets a small team (four to ten people — student project groups, club committees, hackathon teams) create tasks, assign them to members, track status through a defined workflow, receive reminders before deadlines, and see overall progress on a dashboard. It replaces the ad-hoc combination of WhatsApp messages, spreadsheets and memory that such teams currently use. The objective is a single record of truth for who owns what and by when, without the configuration overhead of an enterprise tool like Jira. Full detail on scope is in the synopsis (TM-2026-02_Synopsis.pdf, Section 1.6) and is repeated in 2.2 below.

1.4 References

- TM-2026-02_Synopsis.pdf — Team 2 project synopsis, v1.0, 02 September 2026.
- UE24CS341A Software Engineering Project Guidelines ("Jackfruit"), PES University, 2026.
- IEEE 830 / SRS templates as adapted by the course guidelines.
- draw.io / Jira project artifacts referenced in Appendix C and E.

2. Overall Description

2.1 Product Perspective

TaskFlow is a new, self-contained web product; it is not a follow-on to an existing system. It is a three-tier application: a React single-page application (SPA) in the browser, a Node.js/Express REST API, and a PostgreSQL database, with a third-party e-mail service (sandboxed for development) as the only external dependency (see Section 4 component diagram). It does not integrate with any existing university system.

2.2 Product Functions

At a high level, TaskFlow lets a user: register and log in with role-based access (Member / Owner / Admin); create and manage projects and invite team members; create, edit, assign and track tasks through a defined status workflow, including task dependencies, tags, search and filtering; comment on tasks and see a full activity history; receive in-app and e-mail reminders before a task is due; view a personal "Today" view and a project dashboard with progress, overdue counts and workload; and, for administrators, manage user accounts and review a security audit log. Detailed functional requirements for each of these are in Section 5 (features F1–F8).

2.3 User Classes and Characteristics

User class	Characteristics	Frequency / privilege
Team Member	Non-technical, uses a laptop or mobile browser during work sessions	Frequent use; standard privilege — sees and acts only on tasks/projects they belong to
Project Owner	Non-technical team lead; same devices as Team Member	Frequent use; elevated privilege within their own project(s) only
Administrator	Trained user (typically one of the four team members in a rotating admin role)	Infrequent use; system-wide elevated privilege
E-mail Service (external)	Not a human user — an SMTP sandbox (Mailtrap) in development, a real SMTP provider in production	Invoked by the Notification Service only

2.4 Operating Environment

Server: Node.js 20 LTS runtime with Express, containerized with Docker, deployed to a test environment via the Jenkins CI/CD pipeline. Database: PostgreSQL 16. Client: any evergreen desktop or mobile browser (Chrome, Edge, Firefox, Safari) with JavaScript enabled, at a minimum viewport width of 360px. No native mobile app is provided; the web UI is responsive.

2.5 Design and Implementation Constraints

- Must use Agile/Scrum with visible sprint history in Jira and code history in GitHub (course requirement).
- Must expose a versioned REST API — no breaking changes to a published endpoint once another module depends on it (see API Contract Rules in the architecture/design document).
- Passwords must never be stored or logged in plaintext; only bcrypt hashes are persisted.
- The e-mail service is sandboxed (Mailtrap) for the life of the course project; no real e-mails leave the sandbox.
- Must run inside Docker containers built by the CI pipeline (course requirement for containerization evidence).

2.6 Assumptions and Dependencies

- Assumes each user has exactly one account and a valid, reachable e-mail address for reminders.
- Assumes Postman, Jenkins, SonarQube and Jira remain available for the duration of the project (external tool dependency).
- Assumes team members can dedicate roughly 14 person-days each across the 12-week project (see Appendix E).
- Assumes network connectivity between client and server; TaskFlow does not provide offline support in this version.

3. External Interface Requirements

3.1 User Interfaces

The UI is a responsive React SPA with five primary screen groups: Login/Register, Project & Task Board (Kanban and list views), Task Detail (comments, activity log, dependencies), Dashboard/Today view, and Admin console. Every screen shares a common header (project switcher, notification bell, user menu) and uses a consistent color scheme for task priority and status. Form validation errors are shown inline, next to the field, in red text. Destructive actions (archive project, deactivate user) require a confirmation dialog. Full wireframes will be attached as screenshots in the Deliverable 4 report.

3.2 Software Interfaces

Interface	Description
React SPA ↔ Task API	REST over HTTPS, JSON payloads, JWT bearer token in the Authorization header. See Appendix C for endpoint-to-requirement mapping.
Task/Auth/Project/ Notification Services ↔ PostgreSQL 16	Via Prisma ORM; migrations are version-controlled; connection pooled.
Notification Service ↔ E-mail Service	SMTP via Nodemailer, pointed at a Mailtrap sandbox inbox in development.
CI/CD ↔ GitHub / Jenkins / SonarQube / Docker registry	Webhook-triggered pipeline: lint → unit tests → coverage → SonarQube → Docker build → deploy to test.

3.3 Communications Interfaces

All client-server traffic uses HTTPS (TLS). The API returns standard HTTP status codes (200/201 success, 400 validation error, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict, 500 server error). E-mail reminders are sent as plain-text/HTML MIME e-mails via SMTP. No hardware interfaces apply to this product.

4. Analysis Models

This section presents the use case diagram, domain class diagram, entity-relationship diagram, a representative sequence diagram (Create Task), the task status activity/state diagram, and the system component diagram. All were modeled in draw.io from the same shared design; the live, editable versions are on the team's draw.io account (see Appendix C for the code/requirement cross-reference).

4.1 Use Case Diagram

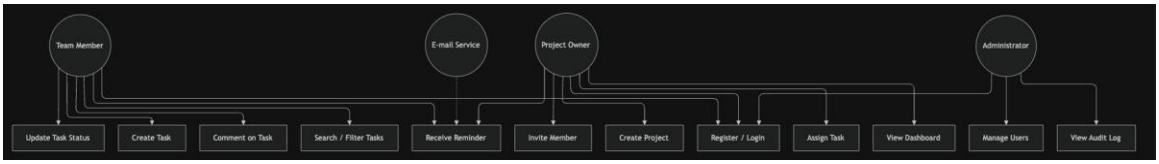


Figure 4.1 — Use case diagram: Team Member, Project Owner and Administrator actors against the 12 primary use cases; the E-mail Service participates in the Receive Reminder use case.

4.2 Domain Class Diagram

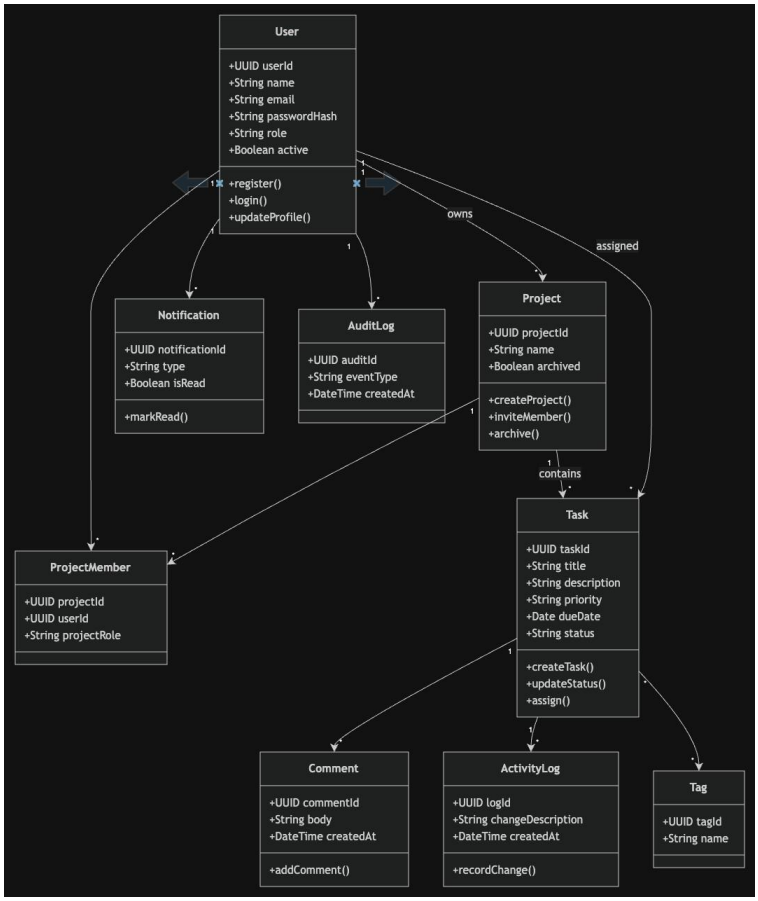


Figure 4.2 — Domain class diagram showing the core entities (User, Project, Task, Tag, Comment, ActivityLog, Notification, AuditLog) and their key operations.

4.3 Entity-Relationship Diagram

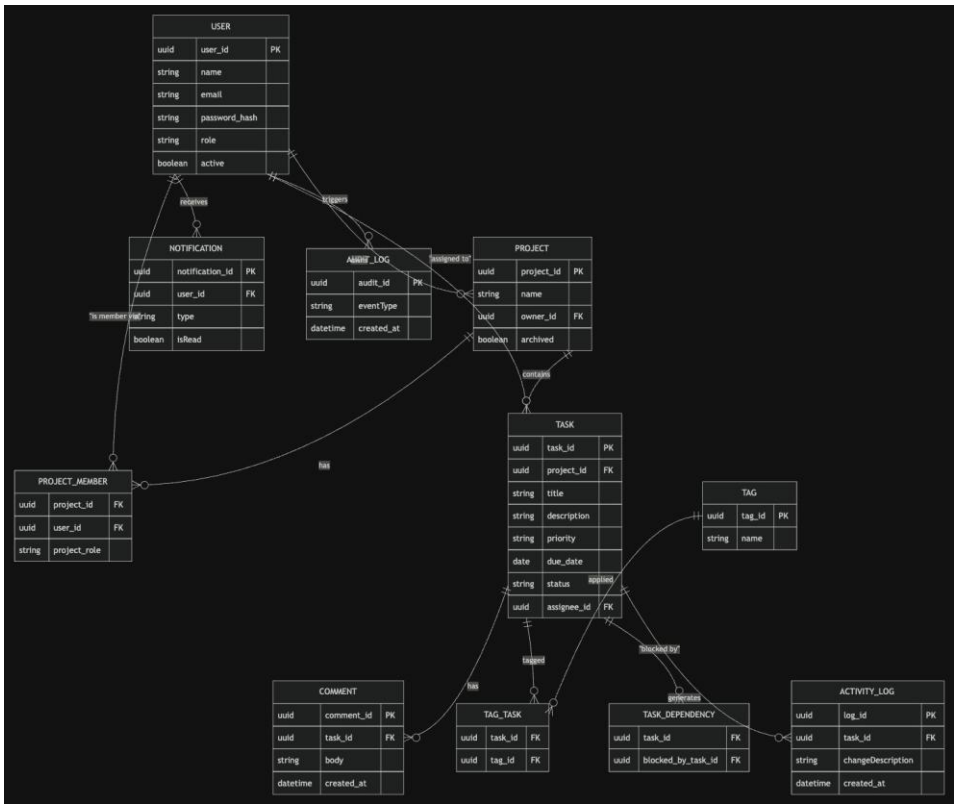


Figure 4.3 — ER diagram for the PostgreSQL schema, including the join tables ProjectMember, TaskTag and TaskDependency.

4.4 Sequence Diagram — Create Task

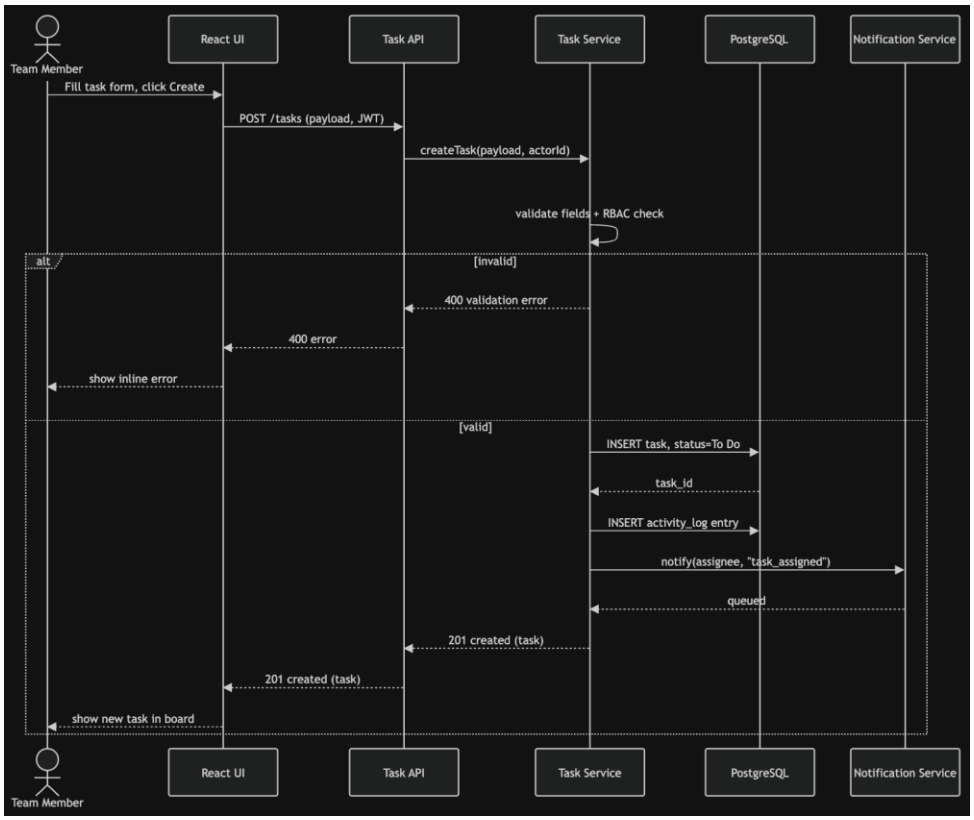


Figure 4.4 — Sequence diagram for UC: Create Task, covering both the validation-failure and success paths through UI → Task API → Task Service → PostgreSQL → Notification Service.

4.5 Activity / State Diagram — Task Status Workflow

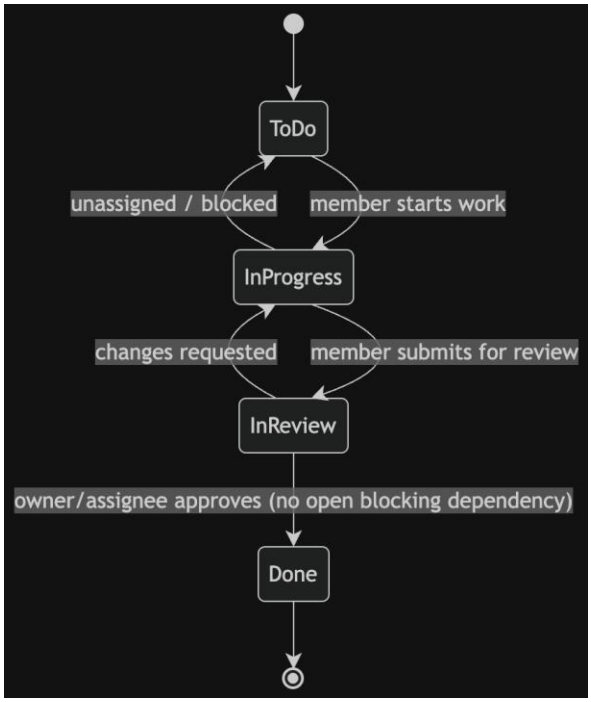


Figure 4.5 — Task status state diagram: To Do → In Progress → In Review → Done, with the review-rejection and unassignment/blocking loops described in F2-REQ-x and the business rules in Section 6.5.

4.6 Component Diagram

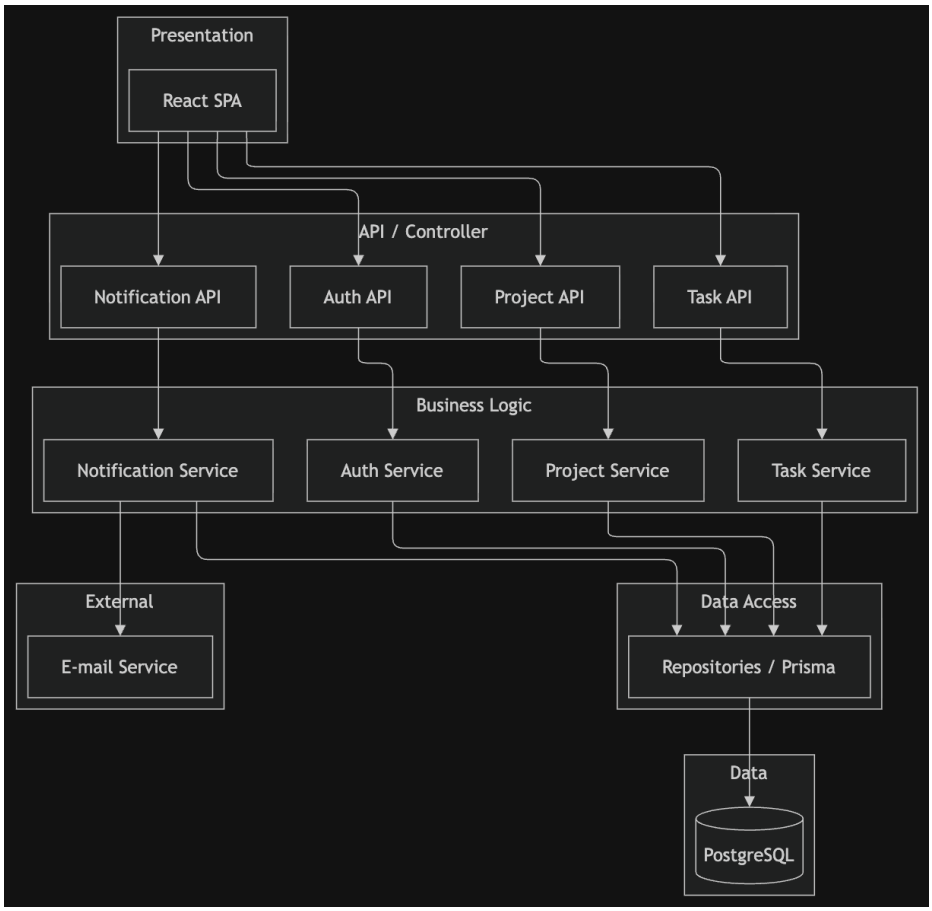


Figure 4.6 — Component diagram: Presentation, API/Controller, Business Logic, Data Access and Data layers, plus the external e-mail service, matching the architecture in the Deliverable 3 design document.

4.7 Use Case Table

UC ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Team Member / Owner / Admin	Application reachable	Authenticated session (JWT issued)
UC-02	Create Project	Project Owner	Logged in	Project created, owner is first member
UC-03	Invite Member	Project Owner	Project exists	Invitation sent; member added on acceptance
UC-04	Create Task	Project Owner / Member	Logged in, member of project	Task created with status To Do
UC-05	Assign Task	Project Owner	Task and target member exist in project	Task assignee updated; notification sent
UC-06	Update Task Status	Assignee / Owner	Task exists, no illegal transition	Task status updated; activity log entry added

UC ID	Use Case	Primary Actor	Precondition	Outcome
UC-07	Comment on Task	Team Member	Task exists, user is project member	Comment stored; mentioned users notified
UC-08	Search / Filter Tasks	Team Member / Owner	Logged in	Filtered task list displayed
UC-09	View Dashboard	Project Owner	Project has ≥ 1 task	Progress, overdue and workload metrics displayed
UC-10	Receive Reminder	Team Member (via E-mail Service)	Task has a due date within the reminder window	In-app notification created; e-mail queued/sent
UC-11	Manage Users	Administrator	Admin authenticated	User activated/deactivated/role reset
UC-12	View Audit Log	Administrator	Admin authenticated	Security-relevant events displayed

5. System Features

Features are organized by the F1–F8 IDs established in the synopsis, kept identical here so the traceability matrix (Appendix C) resolves cleanly. Each requirement is uniquely identified, and is written to be concise, complete, unambiguous, verifiable and necessary, per the SRS template's requirement-writing rule.

5.1 F1: User Accounts and Authentication

Description and Priority

Lets a person register, authenticate and manage their own account, and establishes the Member/Owner/Admin role model enforced on every other feature. Priority: High.

Stimulus/Response Sequences

- User submits the registration form with name, e-mail and password → account is created and a login session begins.
- User submits valid login credentials → a JWT is issued and the user lands on their dashboard.
- User submits an expired or tampered token on any protected request → the API responds 401 and the client redirects to login.

Functional Requirements

Req ID	Requirement
F1-REQ-01	The system shall allow a new user to register with a name, a unique e-mail address and a password of at least 8 characters.
F1-REQ-02	The system shall reject registration when the e-mail address is already registered, returning a clear error without revealing which field is wrong beyond "e-mail already in use".
F1-REQ-03	The system shall store only a bcrypt hash of the password, never the plaintext value, in any datastore or log.
F1-REQ-04	The system shall authenticate a user by e-mail and password and, on success, issue a signed JWT containing the user's ID and role.
F1-REQ-05	The system shall reject a login attempt with invalid credentials with a generic "invalid e-mail or password" message that does not reveal whether the e-mail exists.
F1-REQ-06	The system shall allow an authenticated user to view and update their own name and password.
F1-REQ-07	The system shall attach one role — Member, Owner or Admin — to every user and evaluate that role on every API call to a protected endpoint.
F1-REQ-08	The system shall reject any request to a protected endpoint that lacks a valid, unexpired JWT with an HTTP 401 response.
F1-REQ-09	The system shall reject any request where the authenticated user's role does not permit the requested action with an HTTP 403 response.
F1-REQ-10	The system shall allow a user to log out, invalidating the client-side session token.

5.2 F2: Task Lifecycle Management

Description and Priority

Lets a project member create, edit and progress a task through its lifecycle, including declaring dependencies between tasks. Priority: High.

Stimulus/Response Sequences

- Member fills the task form (title, description, priority, due date) and submits → a task is created with status To Do.
- Assignee moves a task from To Do to In Progress, In Progress to In Review, or In Review to Done → status is updated and an activity log entry is recorded.
- Assignee attempts to mark a task Done while it has an open blocking dependency → the request is rejected with an explanatory error.

Functional Requirements

Req ID	Requirement
F2-REQ-01	The system shall allow a project member to create a task with a title (required), description, priority (Low/Medium/High/Critical), due date and optional time estimate.
F2-REQ-02	The system shall reject task creation when the title is empty or exceeds 200 characters.
F2-REQ-03	The system shall default a newly created task's status to To Do.
F2-REQ-04	The system shall allow a task's status to transition only along the defined path: To Do → In Progress → In Review → Done, with In Review able to return to In Progress on rejection, and In Progress able to return to To Do when unassigned.
F2-REQ-05	The system shall reject a request to set a task's status to Done while the task has at least one incomplete blocked-by dependency, returning an explanatory 409 error.
F2-REQ-06	The system shall allow a project member to declare that Task A is blocked by Task B, and shall reject a dependency declaration that would create a cycle.
F2-REQ-07	The system shall allow the task's owner to edit its title, description, priority, due date and estimate at any time before it reaches Done.
F2-REQ-08	The system shall allow a project member with the appropriate role to delete a task, and shall cascade-delete its comments, activity log entries and tag associations.
F2-REQ-09	The system shall record an activity log entry for every field change and status transition made to a task, capturing the actor, the field, the old and new values and a timestamp.

5.3 F3: Tagging, Search and Filtering

Description and Priority

Lets a user organize and locate tasks quickly across a project using tags, keyword search, filters and two view modes. Priority: Medium.

Stimulus/Response Sequences

- Member types a keyword into the search box → the task list updates to only matching tasks within 1 second for a 1,000-task project.

- Member selects one or more filter chips (status, priority, assignee, tag, due date) → the task list narrows to the intersection of the selected filters.
- Member toggles between Kanban board and list view → the same filtered task set is displayed in the new layout.

Functional Requirements

Req ID	Requirement
F3-REQ-01	The system shall allow a project member to create and apply one or more tags to a task.
F3-REQ-02	The system shall allow a project member to remove a tag from a task without deleting the tag itself if it is used elsewhere.
F3-REQ-03	The system shall support keyword search over task title and description, returning matching tasks within 1 second for a project of 1,000 tasks.
F3-REQ-04	The system shall allow filtering the task list by any combination of status, priority, assignee, tag and due-date range.
F3-REQ-05	The system shall allow sorting the filtered task list by due date, priority or creation date, ascending or descending.
F3-REQ-06	The system shall provide both a Kanban board view (grouped by status) and a list view of the same filtered/sorted task set.
F3-REQ-07	The system shall preserve the active filter and view-mode selection when the user navigates away from and back to the task list within the same session.

5.4 F4: Projects and Team Membership

Description and Priority

Lets a project owner create a project, build its team, and manage project-level lifecycle (archiving). Priority: High.

Stimulus/Response Sequences

- Owner submits the new-project form → a project is created with the owner as its first member.
- Owner invites a member by e-mail → an invitation record is created and the invitee sees it on next login (or receives an e-mail if their address is already registered).
- Invitee accepts an invitation → they are added to the project's member list with the Member role.

Functional Requirements

Req ID	Requirement
F4-REQ-01	The system shall allow an authenticated user to create a project with a name (required, unique per owner) and become that project's Owner.
F4-REQ-02	The system shall allow a project Owner to invite a user to the project by e-mail address.
F4-REQ-03	The system shall allow an invited user to accept or decline a project invitation.
F4-REQ-04	The system shall add an invitee to the project's member list with the Member role immediately on acceptance.
F4-REQ-05	The system shall allow a project Owner to change a member's project role or remove them from the project.
F4-REQ-06	The system shall allow a project Owner to archive a project, after which no new tasks can be created in it but existing data remains viewable.

Req ID	Requirement
F4-REQ-07	The system shall reject any project-management action attempted by a user who is not that project's Owner, returning HTTP 403.

5.5 F5: Assignment, Comments and Activity Log

Description and Priority

Lets an owner assign work to a member and lets the team communicate on a task with a permanent, auditable history. Priority: High.

Stimulus/Response Sequences

- Owner selects an assignee from the project's member list on a task → the task's assignee field is updated and the assignee is notified.
- Member types a comment, optionally with an @mention, and submits → the comment is stored and any mentioned member is notified.
- Member opens a task's activity tab → the full chronological history of changes to that task is displayed.

Functional Requirements

Req ID	Requirement
F5-REQ-01	The system shall allow a project Owner to assign or re-assign a task to any member of that project.
F5-REQ-02	The system shall reject an attempt to assign a task to a user who is not a member of the task's project.
F5-REQ-03	The system shall allow a project member to add a text comment to a task, up to 2,000 characters.
F5-REQ-04	The system shall detect @mentions of project members within a comment and create a notification for each mentioned member.
F5-REQ-05	The system shall display a task's comments in chronological order with the author's name and timestamp.
F5-REQ-06	The system shall display a task's full activity log (status changes, field edits, assignment changes, comments) in chronological order.
F5-REQ-07	The system shall not allow a comment or activity log entry to be edited or deleted once created, to preserve an accurate audit trail.

5.6 F6: Reminders and Notifications

Description and Priority

Keeps users informed of relevant events in-app and, for deadlines, by e-mail — without ever double-sending a reminder. Priority: Medium.

Stimulus/Response Sequences

- A task's due date comes within the configured reminder window (1 day or 3 days before) → an e-mail reminder is sent once and an in-app notification is created.
- A member is assigned a task, mentioned in a comment, or a task they own changes status → an in-app notification is created immediately.
- Member opens the notification bell and marks a notification read → its unread flag is cleared.

Functional Requirements

Req ID	Requirement
F6-REQ-01	The system shall create an in-app notification when a user is assigned a task, mentioned in a comment, or when the status of a task they are assigned to changes.
F6-REQ-02	The system shall send a due-date reminder e-mail to a task's assignee 3 days and again 1 day before the task's due date, provided the task is not yet Done.
F6-REQ-03	The system shall send at most one reminder e-mail per task per configured interval, even if the reminder scheduler runs more than once in that window.
F6-REQ-04	The system shall not send a due-date reminder for a task that has already reached Done status.
F6-REQ-05	The system shall allow a user to mark a single notification, or all notifications, as read.
F6-REQ-06	The system shall display an unread-notification count on the notification bell, updated in real time within the same session.

5.7 F7: Dashboard and Progress Reports

Description and Priority

Gives individuals and project owners an at-a-glance view of workload and progress without manually compiling status. Priority: Medium.

Stimulus/Response Sequences

- Member opens their "Today" view → tasks due today, overdue, and currently in review across all their projects are listed together.
- Owner opens a project's dashboard → completion percentage, a sprint burndown, overdue count and per-member workload are displayed.
- Owner clicks "Export CSV" on a project dashboard → a CSV file of that project's tasks is downloaded.

Functional Requirements

Req ID	Requirement
F7-REQ-01	The system shall provide a personal "Today" view listing, across all of a user's projects, tasks that are due today, overdue, or in the In Review status assigned to them.
F7-REQ-02	The system shall calculate and display a project's completion percentage as (tasks in Done status) / (total non-archived tasks).
F7-REQ-03	The system shall display a sprint burndown chart derived from task counts and due dates for the current sprint.
F7-REQ-04	The system shall display the count of overdue tasks (due date passed, status not Done) for a project.
F7-REQ-05	The system shall display a per-member open-task count (workload) for every member of a project.
F7-REQ-06	The system shall render the project dashboard within 2 seconds for a project containing 500 tasks under the defined test load.
F7-REQ-07	The system shall allow a project Owner to export the current project's task list to a CSV file including title, assignee, status, priority and due date.

5.8 F8: Administration

Description and Priority

Gives a trained administrator the tools to manage user accounts and review security-relevant activity across the whole system. Priority: Medium.

Stimulus/Response Sequences

- Admin opens the user management screen → a list of all users with their status (active/inactive) and role is displayed.
- Admin deactivates a user → that user can no longer log in, but their historical tasks/comments remain intact.
- Admin opens the audit log → security-relevant events across the system are listed in reverse-chronological order.

Functional Requirements

Req ID	Requirement
F8-REQ-01	The system shall allow an Administrator to list all users with their name, e-mail, role and active/inactive status.
F8-REQ-02	The system shall allow an Administrator to deactivate an active user, immediately invalidating that user's existing sessions.
F8-REQ-03	The system shall allow an Administrator to reactivate a previously deactivated user.
F8-REQ-04	The system shall allow an Administrator to change a user's role.
F8-REQ-05	The system shall reject any attempt by a non-Administrator to access user-management or audit-log endpoints with HTTP 403.
F8-REQ-06	The system shall log every failed login attempt, every role change and every account activation/deactivation to the audit log with actor, event type and timestamp.
F8-REQ-07	The system shall allow an Administrator to view the audit log in reverse-chronological order, filterable by event type and date range.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Category	Measurable requirement
NFR-01	Response time	95% of task list, search and filtering API requests shall receive a response within 1 second under the defined test load of 1,000 tasks.
NFR-02	Dashboard rendering	The project dashboard shall render within 2 seconds for a project containing 500 tasks under the defined test load.
NFR-03	Concurrent users	The system shall support at least 20 concurrent users per project while maintaining the response-time target specified in NFR-01 under the defined test workload.

6.2 Safety Requirements

TaskFlow is a productivity tool with no physical, financial or health-safety implications; no specialized safety certification applies. The main "safety" concern is data safety: an operation that could destroy user data (deleting a task, archiving a project) requires explicit confirmation in the UI (see Section 3.1) and is logged in the activity log (F5) or audit log (F8) so it can be traced.

6.3 Security Requirements

ID	Security requirement	Validation
SEC-01	Passwords shall never be stored or logged in plaintext; only a bcrypt hash is persisted.	Code review
SEC-02	Role-based access (Member / Owner / Admin) shall prevent a user from calling an endpoint outside their role's permissions.	Authorization tests
SEC-03	All user-supplied fields shall be validated and sanitized server-side before processing, regardless of client-side validation.	Negative/ security tests
SEC-04	Every protected API endpoint shall reject a request with a missing, expired or invalid JWT with HTTP 401.	Security test
SEC-05	Sensitive fields (password hash, JWT secret) shall never be included in an API response body or client-visible log.	Design/code review
SEC-06	Security-relevant events (failed logins, role changes, account deactivation) shall be recorded in the audit log (F8).	System test
SEC-07	The system shall apply rate limiting to the login endpoint to reduce the effectiveness of credential-stuffing attempts.	Security test

6.4 Software Quality Attributes

Attribute	Requirement
Maintainability	Business logic shall be separated into identifiable service modules (Auth, Project, Task, Notification) with automated unit tests, per the component diagram in Section 4.6.
Testability	Every service module shall expose functions that are independently unit-testable without requiring the full HTTP stack (mockable repository layer).
Usability	A new user shall be able to create a project and its first task in no more than 4 primary screens (per the synopsis's qualitative commitment).

Attribute	Requirement
Reliability	A task creation or reminder creation operation shall not be duplicated when a client retries after a timeout; idempotency keys or an equivalent mechanism shall be used where relevant.
Portability	The application shall run identically in any Docker-compatible container runtime, with no host-specific configuration baked into the image.

6.5 Business Rules

- Only a project's Owner can invite, remove or re-role a member, archive the project, or assign a task.
- Only a task's assignee or the project Owner can change that task's status.
- A task cannot be created without a project; a project cannot be deleted while it has non-archived tasks (it must be archived instead).
- A task cannot be marked Done while it has an open (non-Done) blocking dependency.
- Only Administrators can deactivate/reactivate a user account or change a user's system role.
- A deactivated user's historical data (tasks, comments, activity log entries) is retained and attributed to them, but they cannot log in until reactivated.

7. Other Requirements

7.1 Data Retention

Task, project, comment and activity-log data is retained for the lifetime of the project; there is no automatic deletion. A deactivated user's data is retained per Section 6.5. Audit log entries are retained for the duration of the course project and are not purged.

7.2 Deployment Requirements

The application is deployed via the Jenkins CI/CD pipeline (lint → unit tests → coverage → SonarQube → Docker build → deploy to test environment) as described in the synopsis's tool stack (Section 1.10). No production deployment is in scope for this course project; "deploy to test" targets a containerized test environment reachable by the team and evaluators.

7.3 Legal / Academic Requirements

This is an academic mini-project for UE24CS341A; TaskFlow handles no real personal data beyond what team members and any pilot users voluntarily provide for testing, and the e-mail service remains sandboxed (Mailtrap) throughout, so no real e-mail is sent to any third party.

Appendix A: Glossary

Term	Definition
Task	A unit of work with a title, priority, due date and status, owned by a project and assigned to a member.
Project	A container for tasks and members, owned by exactly one Project Owner.
Role	One of Member, Owner or Admin; determines what actions a user may perform.
RBAC	Role-Based Access Control — restricting API actions based on the caller's role.
JWT	JSON Web Token — a signed token used to authenticate API requests after login.
Blocked-by dependency	A relationship where Task A cannot be marked Done until Task B (which blocks it) is Done.
Activity log	The immutable, chronological record of every change made to a task.
Audit log	The system-wide, immutable record of security-relevant events (failed logins, role changes).
Sandbox e-mail service	A test SMTP endpoint (Mailtrap) that captures outgoing e-mail without delivering it to a real inbox.
Burndown	A chart showing remaining work in a sprint over time.

Appendix B: Field Layouts

Field layouts for the core entities, matching the ER diagram in Section 4.3 and the database design to be finalized in Deliverable 3.

User

Field	Length	Data Type	Description	Mandatory
user_id	36	UUID	Primary key	Y
name	100	String	Display name	Y
email	150	String	Unique login e-mail	Y
password_hash	60	String	bcrypt hash, never plaintext	Y
role	20	Enum	Member / Owner / Admin	Y
active	-	Boolean	False when deactivated by an Admin	Y

Project

Field	Length	Data Type	Description	Mandatory
project_id	36	UUID	Primary key	Y
name	150	String	Project name, unique per owner	Y
owner_id	36	UUID (FK)	References User	Y
archived	-	Boolean	True once archived	Y

Task

Field	Length	Data Type	Description	Mandatory
task_id	36	UUID	Primary key	Y
project_id	36	UUID (FK)	References Project	Y
title	200	String	Task title	Y
description	2000	String	Task description	N
priority	20	Enum	Low / Medium / High / Critical	Y
due_date	-	Date	Deadline	N
status	20	Enum	To Do / In Progress / In Review / Done	Y
assignee_id	36	UUID (FK)	References User	N

Comment / Notification (summary)

Field	Length	Data Type	Description	Mandatory
comment_id / notification_id	36	UUID	Primary key	Y

Field	Length	Data Type	Description	Mandatory
task_id / user_id	36	UUID (FK)	Owning task / target user	Y
body / type	2000 / 30	String	Comment text, or notification type	Y
created_at	-	DateTime	Timestamp	Y

Appendix C: Requirement Traceability Matrix

Complete matrix — all 70 requirement IDs (60 functional across F1–F8, 7 security SEC-01–07, 3 performance NFR-01–03). Architecture references map to the components in Section 4.6. Design (class), Code (file path) and detailed test-case IDs are marked **TBD** and will be populated in Deliverables 3–4 once design classes, code files and test suites exist.

Req	Use Case	Owner	Arch	Design	Code	Verification
F1-REQ-01	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Functional test
F1-REQ-02	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Functional test
F1-REQ-03	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Code review
F1-REQ-04	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Functional test
F1-REQ-05	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Functional test
F1-REQ-06	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Functional test
F1-REQ-07	UC-01	Ankita Pramod	Auth Service (RBAC middleware)	TBD	TBD	Authorization test
F1-REQ-08	UC-01	Ankita Pramod	Auth Service (RBAC middleware)	TBD	TBD	Security test
F1-REQ-09	UC-01	Ankita Pramod	Auth Service (RBAC middleware)	TBD	TBD	Authorization test
F1-REQ-10	UC-01	Ankita Pramod	Auth Service	TBD	TBD	Functional test
F2-REQ-01	UC-04	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-02	UC-04	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-03	UC-04	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-04	UC-06	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-05	UC-06	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-06	UC-04	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-07	UC-04	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-08	UC-04	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F2-REQ-09	UC-06	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F3-REQ-01	UC-08	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F3-REQ-02	UC-08	Ashritha Kamalakar Hegde	Task Service	TBD	TBD	Functional test
F3-REQ-03	UC-08	Ashritha Kamalakar Hegde	Task Service / Task API	TBD	TBD	Functional + performance test
F3-REQ-04	UC-08	Ashritha Kamalakar Hegde	Task Service / Task API	TBD	TBD	Functional test
F3-REQ-05	UC-08	Ashritha Kamalakar Hegde	Task Service / Task API	TBD	TBD	Functional test
F3-REQ-06	UC-08	Ashritha Kamalakar Hegde	Task Service / Task API	TBD	TBD	Functional test
F3-REQ-07	UC-08	Ashritha Kamalakar Hegde	Task Service / Task API	TBD	TBD	Functional test
F4-REQ-01	UC-02	B Nuthan	Project Service	TBD	TBD	Functional test
F4-REQ-02	UC-03	B Nuthan	Project Service	TBD	TBD	Functional test
F4-REQ-03	UC-03	B Nuthan	Project Service	TBD	TBD	Functional test
F4-REQ-04	UC-03	B Nuthan	Project Service	TBD	TBD	Functional test
F4-REQ-05	UC-03	B Nuthan	Project Service	TBD	TBD	Functional test
F4-REQ-06	UC-02	B Nuthan	Project Service	TBD	TBD	Functional test
F4-REQ-07	UC-02	B Nuthan	Project Service	TBD	TBD	Authorization test
F5-REQ-01	UC-05	B Nuthan	Task Service	TBD	TBD	Functional test
F5-REQ-02	UC-05	B Nuthan	Task Service	TBD	TBD	Functional test
F5-REQ-03	UC-07	B Nuthan	Task Service	TBD	TBD	Functional test
F5-REQ-04	UC-07	B Nuthan	Notification Service	TBD	TBD	Functional test
F5-REQ-05	UC-07	B Nuthan	Task Service	TBD	TBD	Functional test
F5-REQ-06	UC-07	B Nuthan	Task Service	TBD	TBD	Functional test

Req	Use Case	Owner	Arch	Design	Code	Verification
F5-REQ-07	UC-07	B Nuthan	Task Service	**TBD**	**TBD**	Functional test
F6-REQ-01	UC-10	Chaithanya B	Notification Service	**TBD**	**TBD**	Functional test
F6-REQ-02	UC-10	Chaithanya B	Notification Service	**TBD**	**TBD**	Functional test
F6-REQ-03	UC-10	Chaithanya B	Notification Service	**TBD**	**TBD**	Reliability test
F6-REQ-04	UC-10	Chaithanya B	Notification Service	**TBD**	**TBD**	Functional test
F6-REQ-05	UC-10	Chaithanya B	Notification Service	**TBD**	**TBD**	Functional test
F6-REQ-06	UC-10	Chaithanya B	Notification Service	**TBD**	**TBD**	Functional test
F7-REQ-01	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Functional test
F7-REQ-02	UC-09	Chaithanya B	Dashboard API / Task Service	**TBD**	**TBD**	Functional test
F7-REQ-03	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Functional test
F7-REQ-04	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Functional test
F7-REQ-05	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Functional test
F7-REQ-06	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Performance test
F7-REQ-07	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Functional test
F8-REQ-01	UC-11	Ankita Pramod	Admin Service	**TBD**	**TBD**	Functional test
F8-REQ-02	UC-11	Ankita Pramod	Admin Service	**TBD**	**TBD**	Functional test
F8-REQ-03	UC-11	Ankita Pramod	Admin Service	**TBD**	**TBD**	Functional test
F8-REQ-04	UC-11	Ankita Pramod	Admin Service	**TBD**	**TBD**	Functional test
F8-REQ-05	UC-11	Ankita Pramod	Admin Service (RBAC middleware)	**TBD**	**TBD**	Authorization test
F8-REQ-06	UC-12	Ankita Pramod	Admin Service (Audit log)	**TBD**	**TBD**	System test
F8-REQ-07	UC-12	Ankita Pramod	Admin Service (Audit log)	**TBD**	**TBD**	Functional test
SEC-01	UC-01	Ankita Pramod	Auth Service	**TBD**	**TBD**	Code review
SEC-02	UC-01	Ankita Pramod	Auth Service (RBAC middleware)	**TBD**	**TBD**	Authorization test
SEC-03	All UCs	All (cross-cutting)	API / Controller layer	**TBD**	**TBD**	Negative / security test
SEC-04	UC-01	Ankita Pramod	Auth Service (RBAC middleware)	**TBD**	**TBD**	Security test
SEC-05	UC-01	Ankita Pramod	Auth Service	**TBD**	**TBD**	Design / code review
SEC-06	UC-12	Ankita Pramod	Admin Service (Audit log)	**TBD**	**TBD**	System test
SEC-07	UC-01	Ankita Pramod	Auth Service	**TBD**	**TBD**	Security test
NFR-01	UC-08	Ashritha Kamalakar Hegde	Task Service / Task API	**TBD**	**TBD**	Performance test
NFR-02	UC-09	Chaithanya B	Dashboard API	**TBD**	**TBD**	Performance test
NFR-03	All UCs	B Nuthan	API / Data layer	**TBD**	**TBD**	Performance test

Appendix D: Initial Validation Specification

At this stage the validation specification is planning-focused: it explains how each requirement category will eventually be validated. This is not the final test report (that is Deliverable 4/5).

Requirement category	Planned validation	Example
Functional (F1–F8)	Black-box functional tests per FR, executed via Postman/Jest	Valid vs. invalid task creation (F2-REQ-01, F2-REQ-02)
Security (SEC-01–07)	Authentication/authorization tests + a manual security pass	Team Member token used to call an Admin-only endpoint → expect 403
Performance (NFR-01–03)	Load/response-time testing against the defined test load	1,000-task project, 20 concurrent users, measure p95 latency
Usability	Task-based walkthrough with a new user	Time and screen-count to create first project + task
Reliability	Failure/retry scenario tests	Reminder scheduler run twice in one window → exactly one e-mail sent
Unit	Automated unit tests + coverage report per service module	Task status transition logic; task dependency validation and dashboard aggregate calculations.

Appendix E: Project Plan

Carried forward from the approved synopsis (TM-2026-02_Synopsis.pdf, Sections 1.8–1.9), repeated here for completeness of the SRS package.

E.1 Sprint Plan

Sprint	Weeks	Planned output	Acceptance evidence	Lead
Sprint 1	Wk 1–2	Project setup (repo, branch strategy, Jenkins skeleton, Docker, DB schema v1), SRS baseline, F1 registration/login	User stories + Git repo + working login	Ankita, all
Sprint 2	Wk 3–4	F2 task CRUD + status workflow; F4 projects and membership; architecture + UML	Demo + unit tests + design docs	Ashritha, Nuthan
Sprint 3	Wk 5–6	F3 tags/search/filter with Kanban view; F5 assignment, comments, activity log; RBAC on all endpoints	Integration tests + PRs + Postman collection	Ashritha, Nuthan
Sprint 4	Wk 7–8	F6 notifications + e-mail reminders; F7 Today view and project dashboard	System demo + test evidence + coverage ≥ 80% lines	Chaithanya
Sprint 5	Wk 9–10	F8 administration + audit log; security hardening; SonarQube clean-up; performance test	Validation + coverage + security report	Ankita, all
Sprint 6	Wk 11–12	System validation with pilot users, defect fixing, maintenance plan, final report and presentation	Final report + deployment/demo	All

E.2 Work Breakdown and Effort Estimate

Work package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements, use cases, validation planning	All	6	SRS, validation spec v1
WP2	Architecture, UML, API contract, DB design	All (Nuthan leads DB)	5	Architecture + design docs
WP3	F1 + F8 auth, RBAC, admin	Ankita	8	Working features + unit tests
WP4	F2 + F3 task lifecycle, tags, search	Ashritha	8	Working features + unit tests
WP5	F4 + F5 projects, assignment, comments	Nuthan	8	Working features + unit tests
WP6	F6 + F7 notifications, dashboard, reports	Chaithanya	8	Working features + unit tests
WP7	CI/CD, Docker, SonarQube, security testing	All (Ankita leads)	6	Pipeline + security evidence

Work package	Tasks	Owner	Effort (person-days)	Output
WP8	System validation, final report, presentation	All	6	Final report, demo

Total estimated effort: ~55 person-days (~14 per member across 12 weeks). A Gantt chart derived from this WBS is tracked live on the team's Jira board (project TAS, epic TAS-1 "TaskFlow – SE Mini-Project", stories TAS-2 through TAS-9 for features F1–F8).

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Mini-Project — Phase 1

Software Architecture and Design Specification

for

TaskFlow — Collaborative Task Manager Application

Version 1.0 – Draft for Review

Prepared by: Ankita Pramod, Ashritha Kamalakar Hegde, B Nuthan, Chaithanya B (Team 2)

PES University, Bangalore — UE24CS341A Software Engineering

03 October 2026

Item	Entry
Project ID	TM-2026-02
Group Number	Team 2
Course	UE24CS341A — Software Engineering
Technology	JavaScript — React.js (frontend) + Node.js/Express (backend), PostgreSQL
Status	Draft
Related Documents	TM-2026-02_Synopsis.pdf (Deliverable 1), TM-2026-02_SRS.pdf (Deliverable 2)

Revision History

Version	Date	Author	Change Summary
0.1	02 Sep 2026	Team 2	Initial component/architecture sketch (from SRS Section 4.6)
1.0	03 Oct 2026	Team 2	First complete SAD for combined SRS+SAD+STP submission

Approvals

Role	Name	Signature/Date
Team Lead	Ankita Pramod	
Faculty Reviewer		

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of Version 1.0 of TaskFlow, a collaborative task manager application built by Team 2 (Project ID TM-2026-02) for the UE24CS341A Software Engineering mini-project. It translates the functional and non-functional requirements captured in the SRS (Section 5, features F1–F8, and Section 6) into a concrete system structure: components, their responsibilities, the chosen architecture pattern, the data design, the API contracts between components, and the security architecture.

1.2 Scope

Covers the complete first release of TaskFlow: user accounts and authentication, task lifecycle management, tagging/search/filtering, projects and team membership, assignment/comments/activity log, reminders and notifications, dashboards and progress reports, and administration (F1–F8). It does not cover a native mobile app, offline support, or integration with any existing university system — all out of scope per SRS Sections 1.3 and 2.6.

1.3 Audience

The four development team members (as implementers), course faculty/evaluators (for design assessment and traceability), and any future maintainer of the codebase.

1.4 Definitions

RBAC, JWT, Blocked-by dependency, Activity log, Audit log, Sandbox e-mail service — as defined in the SRS Appendix A Glossary, repeated here for convenience: RBAC (Role-Based Access Control — restricting API actions based on the caller's role); JWT (JSON Web Token — a signed token used to authenticate API requests after login); Blocked-by dependency (a relationship where Task A cannot be marked Done until Task B, which blocks it, is Done); Activity log (the immutable, chronological record of every change made to a task); Audit log (the system-wide, immutable record of security-relevant events).

2. Document Overview

2.1 How to use this document

Section 3 presents the architecture: the goals and constraints that shaped it, the component (UML) diagram, a description of every component, the chosen architecture pattern and its rationale, the technology stack, architectural risks, traceability from requirements to components, and the security architecture (STRIDE threat model). Section 4 presents the detailed design: two UML sequence diagrams, the REST API design, error handling/logging/monitoring conventions, the data design (domain class diagram and ER diagram), and UX design notes. Section 5 holds the appendices.

2.2 Related Documents

TM-2026-02_Synopsis.pdf (Deliverable 1); TM-2026-02_SRS.pdf (Deliverable 2, Sections 4–6 and Appendix C in particular); the combined Software Test Plan (STP) that follows this document in the same submission.

3. Architecture

3.1 Goals & Constraints

Goals: a maintainable, testable three-tier web application that a four-person student team can build and demo within a 12-week, 6-sprint schedule; clear separation between presentation, API, business logic and data access so each of the four features owners (Section 3.8) can work independently; secure-by-default handling of credentials and role-based access.

- Constraint: must use Agile/Scrum with visible sprint history in Jira and code history in GitHub (course requirement).
- Constraint: must expose a versioned REST API — no breaking changes to a published endpoint once another module depends on it.
- Constraint: passwords must never be stored or logged in plaintext; only bcrypt hashes are persisted.
- Constraint: the e-mail service is sandboxed (Mailtrap) for the life of the course project; no real e-mails leave the sandbox.
- Constraint: must run inside Docker containers built by the CI pipeline (course requirement for containerization evidence).

3.2 Stakeholders & Concerns

Stakeholder	Primary Concern
Team Member / Project Owner (end users)	Usability, response time, data not being lost or duplicated
Course faculty / evaluators	Architectural clarity, requirement traceability, security hygiene
Development team (Team 2)	Modularity — ability to build/test F1–F8 somewhat independently per owner
Administrator (system role)	Visibility into security-relevant events (audit log), ability to deactivate misbehaving accounts

3.3 Component (UML) Diagram

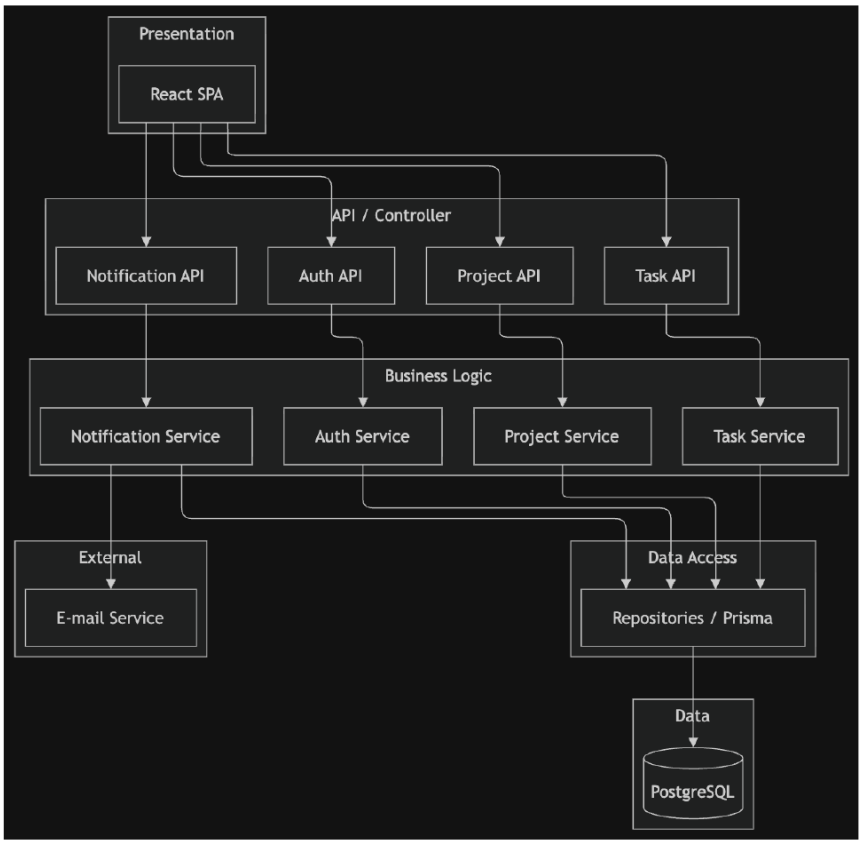


Figure 4.6 — Component diagram: Presentation, API/Controller, Business Logic, Data Access and Data layers, plus the external e-mail service, matching the architecture in the Deliverable 3 design document.

Figure 3.1 — Component diagram: Presentation, API/Controller, Business Logic, Data Access and Data layers, plus the external e-mail service.

3.4 Component Descriptions

Component	Layer	Responsibility
React SPA	Presentation	Single-page application; renders Login/Register, Project & Task Board (Kanban + list), Task Detail, Dashboard/Today view, and Admin console. Holds the JWT client-side and attaches it as a bearer token on every API call.
Auth API / Auth Service	API → Business Logic	Registration, login, logout, profile update; issues and validates JWTs; evaluates a user's role (Member/Owner/Admin) via RBAC middleware shared by every other service.
Project API / Project Service	API → Business Logic	Project creation, member invitation/acceptance, role changes, archiving; enforces that only a project's Owner can manage it (F4-REQ-07).
Task API / Task Service	API → Business Logic	Task CRUD, status-workflow transitions, dependency

		declarations, assignment, tagging, search/filter, and the activity log (F2, F3, F5).
Notification API / Notification Service	API → Business Logic	In-app notifications, @mention detection, e-mail reminder scheduling/de-duplication, and the unread-count badge (F6).
Repositories / Prisma	Data Access	ORM layer; one repository per aggregate (User, Project, Task, Notification, AuditLog); owns all SQL and migrations, pooled connections.
PostgreSQL 16	Data	System of record for all entities in Section 4.3 of the SRS ER diagram.
E-mail Service (external)	External	SMTP endpoint (Mailtrap sandbox in development) invoked only by the Notification Service.

3.5 Chosen Architecture Pattern and Rationale

A layered (N-tier) architecture was chosen — Presentation → API/Controller → Business Logic → Data Access → Data — matching Figure 3.1. Each horizontal layer only calls the layer directly below it, which keeps the dependency graph acyclic and lets each of the four team members own one vertical slice (one API + one service) without needing to understand the others' internals in depth.

- Layered architecture was chosen over microservices: four independently deployable services would add operational overhead (service discovery, inter-service auth, distributed tracing) that is disproportionate to a single-team, 12-week course project with one shared PostgreSQL instance.
- Layered architecture was chosen over a monolithic MVC-with-no-service-layer design because the course requires unit-testable business logic (SRS NFR — Testability, Section 6.4): a distinct Business Logic layer keeps service functions independently unit-testable without the full HTTP stack, using a mockable repository layer.
- A single shared PostgreSQL database (rather than per-feature datastores) was chosen because TaskFlow's entities are heavily relational (Task → Project → User, with join tables for membership, tags and dependencies) and a 12-week project does not justify the complexity of keeping several datastores consistent.

3.6 Technology Stack & Data Stores

Layer	Technology
Client	React.js single-page application; any evergreen desktop/mobile browser, minimum viewport 360px
API / Controller	Node.js 20 LTS with Express, REST over HTTPS, JSON payloads, JWT bearer auth
Business Logic	Plain JavaScript service modules (Auth, Project, Task, Notification), independently unit-tested with Jest
Data Access	Prisma ORM, version-controlled migrations, pooled connections
Data store	PostgreSQL 16
External	SMTP via Nodemailer, pointed at a Mailtrap sandbox inbox in development
Deployment	Docker containers, Jenkins CI/CD pipeline (lint → unit tests → coverage → SonarQube → Docker build → deploy to test)

3.7 Risks & Mitigations

Risk	Mitigation
A single shared PostgreSQL instance becomes a bottleneck or single point of failure under the 20-concurrent-user test load (NFR-03)	Connection pooling via Prisma; indexes on project_id/status/assignee_id (the columns F3's search/filter queries use); load-tested against the defined test load before Sprint 5 sign-off
Circular or deep blocked-by task dependencies corrupt the status workflow (F2-REQ-06)	Cycle check enforced in the Task Service before a dependency is persisted; covered by dedicated unit tests
A team member's unfamiliarity with Prisma/RBAC middleware slows a feature's delivery	Auth Service (with the shared RBAC middleware) is built first in Sprint 1 so all other features can depend on a working, tested contract
Reminder e-mails sent twice if the scheduler runs more than once in a window (F6-REQ-03)	Idempotency: one reminder row per (task, interval) with a unique constraint, checked before a reminder e-mail is queued

3.8 Traceability to Requirements

Full requirement-to-architecture traceability (all 70 requirement IDs) is maintained in the SRS Appendix C Requirement Traceability Matrix; it is not repeated in full here to avoid divergence between documents. The feature-to-component mapping that drove that matrix's “Arch” column is:

Feature	Owner	Primary Component(s)
F1 — User Accounts and Authentication	Ankita Pramod	Auth API / Auth Service (+ RBAC middleware, shared by all services)
F2 — Task Lifecycle Management	Ashritha Kamalakar Hegde	Task API / Task Service
F3 — Tagging, Search and Filtering	Ashritha Kamalakar Hegde	Task API / Task Service
F4 — Projects and Team Membership	B Nuthan	Project API / Project Service
F5 — Assignment, Comments and Activity Log	B Nuthan	Task API / Task Service
F6 — Reminders and Notifications	Chaithanya B	Notification API / Notification Service, E-mail Service
F7 — Dashboard and Progress Reports	Chaithanya B	Dashboard API (reads via Task Service / Project Service)
F8 — Administration	Ankita Pramod	Admin Service (Auth API) / Audit log

3.9 Security Architecture

Threat modeled using STRIDE against the component diagram in Figure 3.1:

Threat (STRIDE)	Where	Mitigation
Spoofing	Login, any protected endpoint	JWT bearer auth (SEC-04); generic “invalid e-mail or password” error that does not reveal which field is wrong (F1-REQ-05); rate limiting on login (SEC-07)
Tampering	Task/Project writes, status transitions	Server-side validation of every field regardless of client-side validation (SEC-03); status transitions restricted to the defined workflow (F2-REQ-04); activity log and audit log are append-only (F5-REQ-07)
Repudiation	Any state-changing action	Every task field/status change is written to the activity log with actor, field, old/new value and

		timestamp (F2-REQ-09); every security-relevant event is written to the audit log (F8-REQ-06)
Information Disclosure	API responses, logs	Password hash and JWT secret never included in an API response body or client-visible log (SEC-05); only a bcrypt hash is persisted, never plaintext (SEC-01)
Denial of Service	Login endpoint, search endpoint	Rate limiting on the login endpoint (SEC-07); indexed, bounded search queries sized to the defined test load
Elevation of Privilege	Project-management and Admin endpoints	RBAC middleware evaluates the caller's role on every protected endpoint and rejects with 403 if not permitted (F1-REQ-07/09, F4-REQ-07, F8-REQ-05, SEC-02)

4. Design

4.1 Design Overview

TaskFlow is designed as layered modules that mirror the architecture in Section 3: an API/controller layer that only parses requests, checks auth/role and calls a service; a business-logic layer that owns all validation and workflow rules and is unit-tested without the HTTP stack; and a data-access layer that owns all persistence. This keeps each layer independently testable and lets changes to one layer (e.g. swapping the ORM) avoid touching the others.

4.2 UML Sequence Diagrams

Two representative flows are shown: Create Task (the core write path exercised by F2) and Assign Task (the cross-feature path that exercises F5's assignment rule together with F6's notification rule).

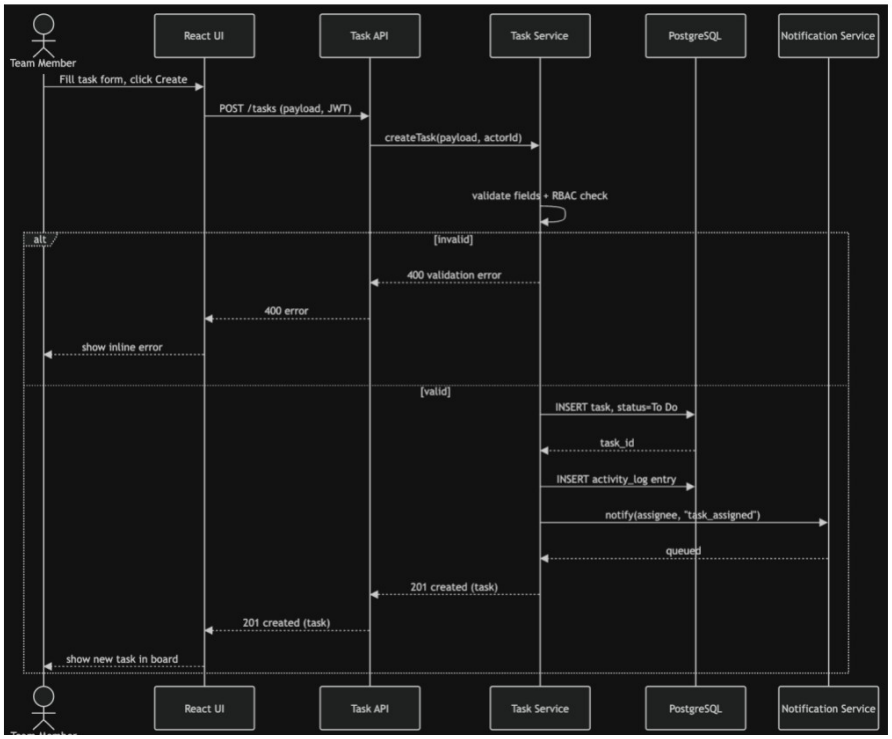


Figure 4.1 — Sequence diagram for UC-04: Create Task, covering both the validation-failure and success paths through UI → Task API → Task Service → PostgreSQL → Notification Service.

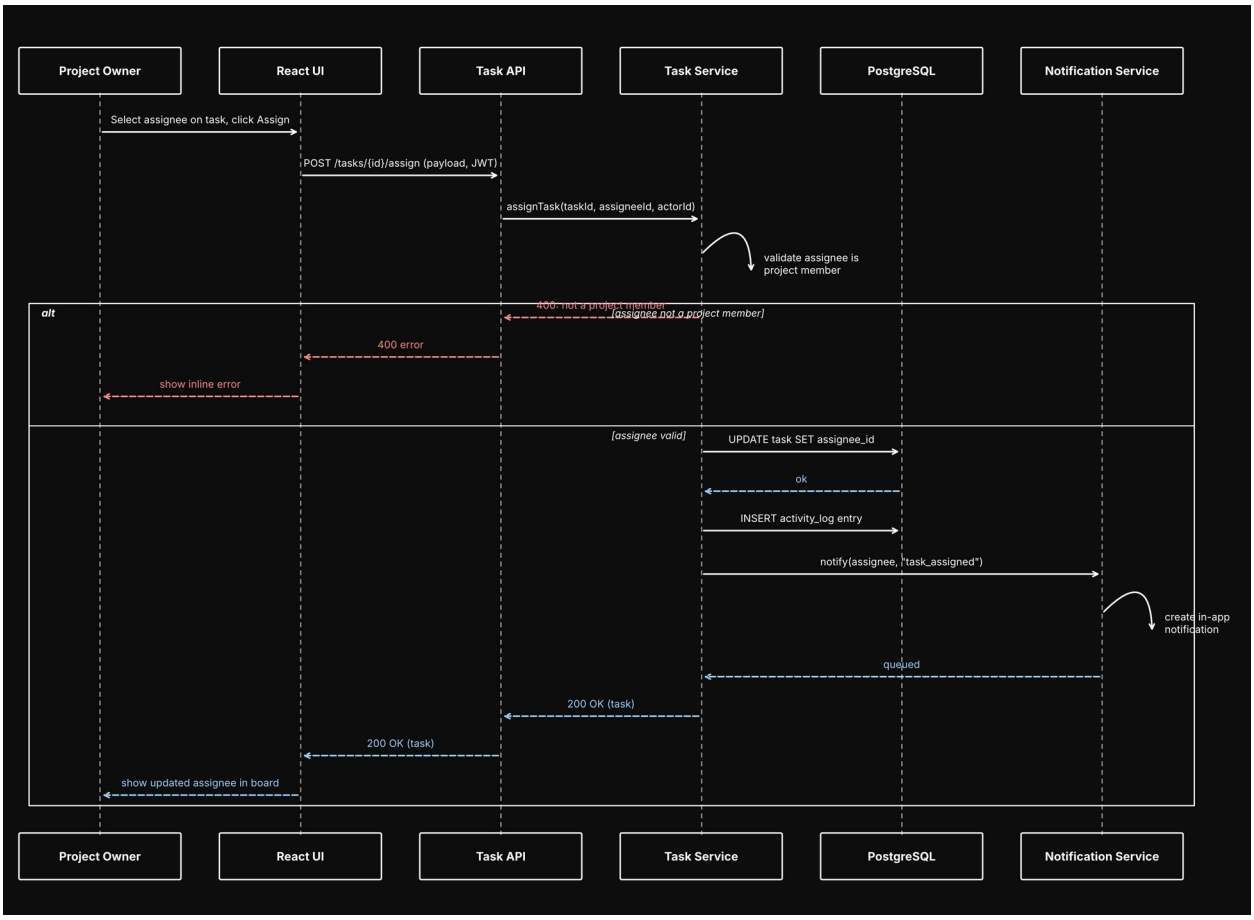


Figure 4.2 — Sequence diagram for UC-05: Assign Task, covering the membership-validation failure path and the success path through Task API → Task Service → PostgreSQL → Notification Service.

4.3 API Design

Interface definitions for the Task API and the Auth API (the two components most other flows depend on); the remaining APIs (Project, Notification) follow the same JSON/JWT/HTTP-status conventions.

Endpoint	Method	Request	Response	Errors
/auth/register	POST	{name, email, password}	{userId, token}	400 validation error, 409 e-mail already in use (F1-REQ-02)
/auth/login	POST	{email, password}	{token, role}	401 invalid e-mail or password (F1-REQ-05)
/tasks	POST	{projectId, title, description, priority, dueDate}, JWT	201 {task}	400 validation error (title empty/>200 chars), 401 unauthenticated
/tasks/{id}/status	PATCH	{status}, JWT	200 {task}	409 illegal transition or open blocking dependency (F2-REQ-04/05)
/tasks/{id}/assign	POST	{assigneeId}, JWT	200 {task}	400 assignee not a

				project member (F5-REQ-02), 403 not project Owner
/tasks?search=&status=&priority=&assignee=&tag=	GET	query params, JWT	200 {tasks[]}	401 unauthenticated

4.4 Error Handling, Logging & Monitoring

All APIs return the standard HTTP status codes defined in SRS Section 3.3 (200/201 success, 400 validation error, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict, 500 server error), each with a JSON body of the form {error: {code, message}}. No sensitive field (password hash, JWT secret) is ever included in a response body or written to a log (SEC-05). Every API request is logged with method, path, status code and latency (no request/response bodies) for monitoring; the activity log and audit log (Sections 3.9 and SRS F5/F8) serve as the application-level audit trail. Monitoring targets for the course deployment: request latency against NFR-01/NFR-02, and failed-login rate as an early signal of credential-stuffing attempts (SEC-07).

4.5 Data Design

The domain class diagram and the PostgreSQL entity-relationship diagram (carried forward unchanged from SRS Sections 4.2 and 4.3) are the authoritative data design for this release.

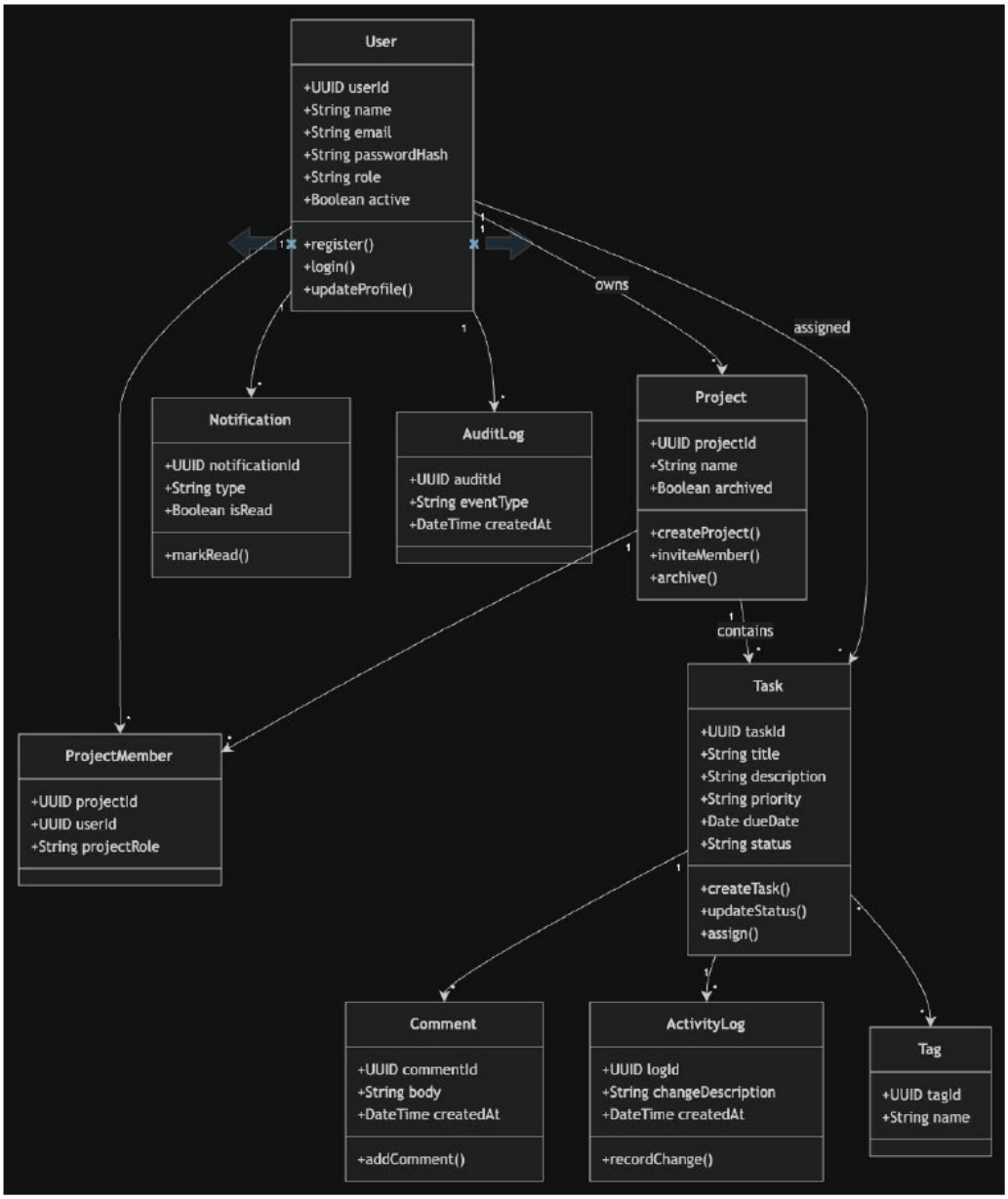


Figure 4.3 — Domain class diagram.

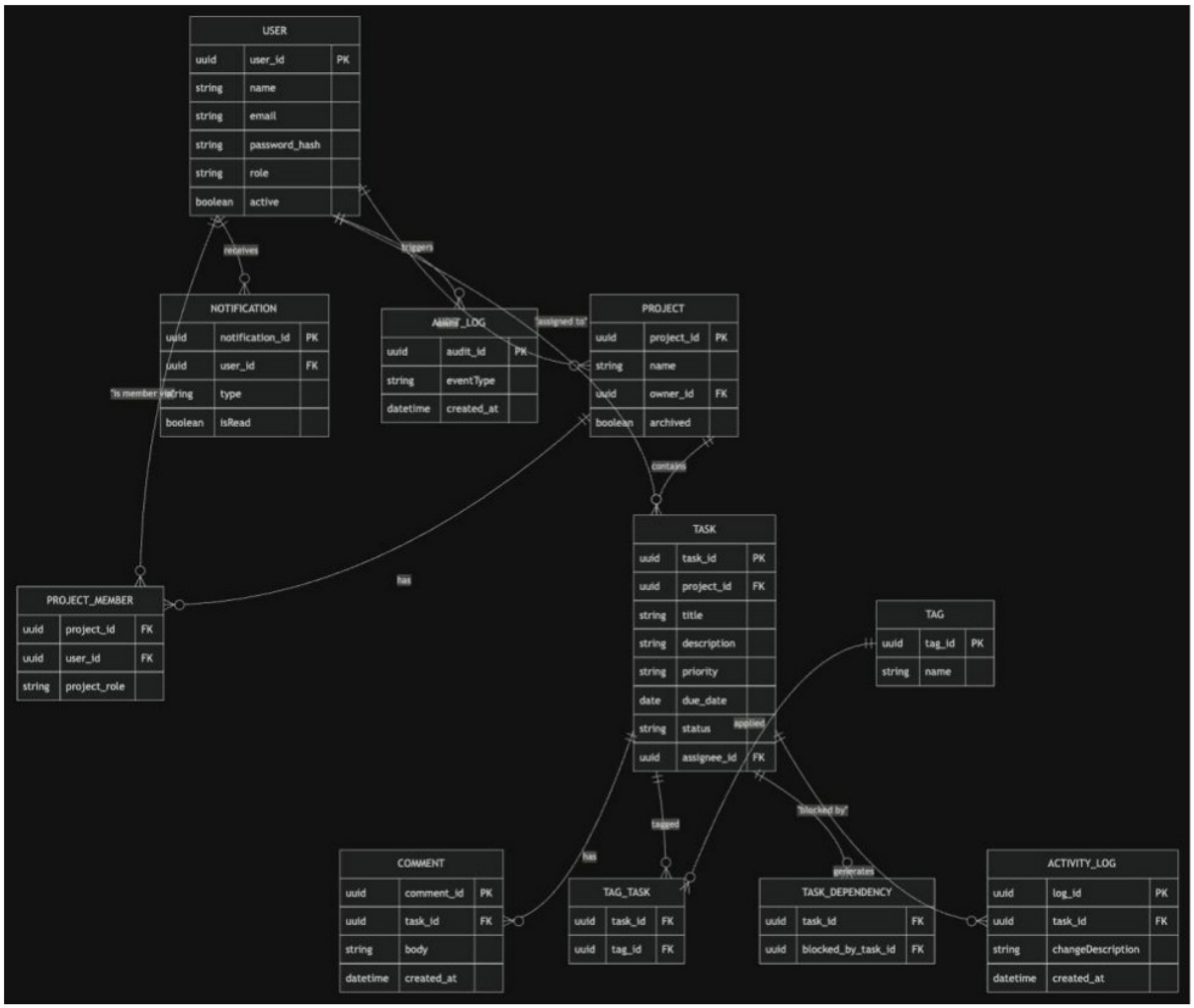


Figure 4.4 — ER diagram for the PostgreSQL schema, including the join tables ProjectMember, TaskTag and TaskDependency.

4.6 UX Design

The UI is a responsive React SPA with five primary screen groups — Login/Register, Project & Task Board (Kanban and list views), Task Detail (comments, activity log, dependencies), Dashboard/Today view, and Admin console — sharing a common header (project switcher, notification bell, user menu) and a consistent color scheme for task priority and status. Form validation errors are shown inline, next to the field, in red text. Destructive actions (archive project, deactivate user) require a confirmation dialog, per SRS Section 3.1. A new user can create a project and its first task in no more than four primary screens (SRS Section 6.4, Usability).

4.7 Open Issues & Next Steps

- Design and Code columns of the SRS Appendix C traceability matrix are marked TBD and will be populated once design classes and file paths exist (Deliverable 3/4).

- Full wireframes for all five screen groups will be attached as screenshots in the Deliverable 4 report.
- Rate-limiting thresholds for SEC-07 (login endpoint) are not yet numerically fixed; to be set during Sprint 5 security hardening.

5. Appendices

5.1 Glossary

See SRS Appendix A for the full glossary (Task, Project, Role, RBAC, JWT, Blocked-by dependency, Activity log, Audit log, Sandbox e-mail service, Burndown).

5.2 References

- TM-2026-02_SRS.pdf — Team 2 Software Requirements Specification, v1.0.
- UE24CS341A Software Engineering Project Guidelines (“Jackfruit”), PES University, 2026.
- IEEE 42010 (architecture description), as adapted by the course's SAD template.

5.3 Tools

draw.io (component, class, ER and state diagrams); Python/matplotlib (sequence diagrams); Jira (sprint and backlog tracking, project TAS); GitHub + Jenkins (CI/CD).

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Mini-Project — Phase 1

Software Test Plan

for

TaskFlow — Collaborative Task Manager Application

Version 1.0 – Draft for Review

Prepared by: Ankita Pramod, Ashritha Kamalakar Hegde, B Nuthan, Chaithanya B (Team 2)

PES University, Bangalore — UE24CS341A Software Engineering

03 October 2026

Item	Entry
Project ID	TM-2026-02
Group Number	Team 2
Status	Draft
Related Documents	TM-2026-02_SRS.pdf (Deliverable 2), TM-2026-02_SAD.pdf (this submission)

1. Introduction

Purpose: This document defines the test plan for TaskFlow v1.0. It outlines objectives, scope, strategy, resources, schedule and responsibilities for testing, and maps every SRS requirement (F1–F8 functional, NFR-01–03 performance, SEC-01–07 security) to concrete test cases.

Scope: Testing covers all eight TaskFlow features — user accounts and authentication, task lifecycle, tagging/search/filtering, projects and membership, assignment/comments/activity log, reminders and notifications, dashboards, and administration — plus the non-functional and security requirements in SRS Section 6. The underlying Docker/Jenkins CI infrastructure and third-party tools (Mailtrap, PostgreSQL itself) are exercised through TaskFlow but are not independently tested.

References: TM-2026-02_SRS.pdf v1.0; TM-2026-02_SAD.pdf (this submission); UE24CS341A Software Engineering Project Guidelines (“Jackfruit”), PES University, 2026.

Definitions: SRS (Software Requirements Specification), RTM (Requirement Traceability Matrix), RBAC (Role-Based Access Control), JWT (JSON Web Token), UAT (User Acceptance Testing). See SRS Appendix A for the full project glossary.

2. Test Items

- Auth module (registration, login, logout, JWT issuance/validation, RBAC middleware) — F1
- Task lifecycle module (create/edit/delete, status workflow, dependencies) — F2
- Tagging/search/filter module — F3
- Project and membership module — F4
- Assignment, comments and activity-log module — F5
- Notification and reminder module (in-app + e-mail) — F6
- Dashboard and reporting module — F7
- Administration module (user management, audit log) — F8

3. Features to be Tested

Features mapped to SRS requirement IDs (full requirement text is in SRS Section 5–6):

Feature	Requirement IDs
F1 — User Accounts and Authentication	F1-REQ-01 .. F1-REQ-10
F2 — Task Lifecycle Management	F2-REQ-01 .. F2-REQ-09
F3 — Tagging, Search and Filtering	F3-REQ-01 .. F3-REQ-07
F4 — Projects and Team Membership	F4-REQ-01 .. F4-REQ-07
F5 — Assignment, Comments and Activity Log	F5-REQ-01 .. F5-REQ-07
F6 — Reminders and Notifications	F6-REQ-01 .. F6-REQ-06
F7 — Dashboard and Progress Reports	F7-REQ-01 .. F7-REQ-07
F8 — Administration	F8-REQ-01 .. F8-REQ-07
Performance (non-functional)	NFR-01, NFR-02, NFR-03
Security	SEC-01 .. SEC-07

4. Features Not to be Tested

- Native mobile app — out of scope for this release (SRS Section 2.4).
- Offline support — explicitly out of scope (SRS Section 2.6).
- Third-party SMTP delivery beyond the Mailtrap sandbox — real e-mail delivery is never exercised (SRS Section 7.3).
- Jenkins/Docker/SonarQube internals — used as infrastructure, not test subjects in their own right.

5. Test Approach / Strategy

Levels:

- Unit tests (module-level): each business-logic service (Auth, Project, Task, Notification) tested in isolation against a mocked repository layer, per SRS NFR — Testability.
- Integration tests: API endpoint → service → PostgreSQL, run against a test database via a Postman collection / Jest + Supertest.
- System tests: end-to-end TaskFlow flows through the React UI (e.g. register → create project → create task → assign → complete).
- Acceptance tests (UAT): a short walkthrough with pilot users per SRS Appendix D (Usability row).

Types:

- Functional testing — covers F1–F8 per the Req-ID-mapped test cases in Section 13.
- Regression testing — the full Postman collection re-run on every pull request via the Jenkins pipeline.
- Performance testing — response time and concurrency per NFR-01/02/03.
- Security testing — Section 5.1 below.
- Usability testing — screen-count/time-to-first-task walkthrough per SRS Appendix D.

Entry Criteria: A stable build is deployed to the Dockerized test environment via the Jenkins pipeline; the test database is seeded; the Postman/Jest test collection is up to date with the current API contract.

Exit Criteria: 100% of the planned test cases in Section 13 executed; 0 critical (Sev-1) defects open; unit test line coverage ≥ 80% per service module (SRS Appendix E, Sprint 4 acceptance evidence); all SEC-01..07 security test cases pass.

5.1 Security Validation

- Authentication/authorization tests: every protected endpoint rejects a missing/expired/invalid JWT with 401 (SEC-04), and rejects a role that does not permit the action with 403 (SEC-02).
- Password handling: confirm only a bcrypt hash is ever persisted or logged, never plaintext (SEC-01); confirm the password hash and JWT secret never appear in an API response body (SEC-05).
- Input validation / negative testing: server-side validation and sanitization of all user-supplied fields, independent of client-side validation (SEC-03) — e.g. oversized title, SQL-metacharacter injection attempts in search, malformed e-mail.

- Audit trail: confirm failed logins, role changes and account activation/deactivation are recorded in the audit log (SEC-06).
- Rate limiting: confirm repeated failed login attempts against the same account are throttled (SEC-07).
- A manual security pass (code review of the Auth Service and RBAC middleware) is performed in Sprint 5 alongside the automated security tests, per SRS Appendix D.

6. Test Environment

Item	Detail
Runtime	Node.js 20 LTS / Express, containerized with Docker, deployed to a test environment via the Jenkins CI/CD pipeline
Database	PostgreSQL 16, seeded test dataset (includes a 1,000-task project for NFR-01 and a 500-task project for NFR-02)
Client	Any evergreen desktop/mobile browser (Chrome, Edge, Firefox, Safari), minimum viewport 360px
E-mail	Mailtrap sandbox inbox (no real e-mail leaves the sandbox)
Tools	Postman (API/functional + security tests), Jest + Supertest (unit/integration), JMeter or autocannon (performance/load), Jira (defect tracking, project TAS), SonarQube (static analysis / code smells)
Test Data	Seeded users for each role (Member, Owner, Admin); seeded projects/tasks covering every status and priority value

7. Test Schedule

Carried forward from the SRS Appendix E sprint plan (Section 1.9 of the synopsis) — testing is embedded in every sprint rather than deferred to the end:

Sprint	Weeks	Testing Milestone
Sprint 1	Wk 1–2	Test plan skeleton + F1 (auth) unit tests alongside the working login
Sprint 2	Wk 3–4	F2/F4 integration tests; architecture/design docs (this SAD) finalized
Sprint 3	Wk 5–6	F3/F5 integration tests; Postman collection complete for F1–F5
Sprint 4	Wk 7–8	F6/F7 system tests; coverage ≥ 80% lines gate enforced in CI
Sprint 5	Wk 9–10	F8 tests; full SEC-01..07 security pass; NFR-01..03 performance tests
Sprint 6	Wk 11–12	UAT with pilot users; regression pass; defect fixing; final test summary report

8. Test Deliverables

- This Test Plan document

- Test cases (Section 13) — manual (Postman) and automated (Jest)
- Postman collection and Jest test scripts
- Seeded test data / fixtures
- Test execution logs (Jenkins pipeline output)
- Defect reports (Jira, project TAS)
- Final Test Summary Report (Deliverable 4/5)

9. Roles and Responsibilities

Role	Name	Responsibility
QA / Test Lead	Ankita Pramod	Owns this test plan; leads security and auth testing (F1, F8, SEC-01..07)
Test Engineer	Ashritha Kamalakar Hegde	Designs and executes F2/F3 test cases; performance testing (NFR-01)
Test Engineer	B Nuthan	Designs and executes F4/F5 test cases; performance testing (NFR-03)
Test Engineer	Chaithanya B	Designs and executes F6/F7 test cases; performance testing (NFR-02)
Product Owner (course faculty)	—	Approves test results, signs off release readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in a stable build reaching the test environment before a sprint demo	Jenkins pipeline builds and deploys to test on every merge to main; smoke tests requested early from the feature owner
Test environment downtime (shared Dockerized test host)	Docker Compose config kept in the repo so any team member can stand up a local copy on short notice
Dependency on the Mailtrap sandbox for reminder/e-mail tests	Reminder logic unit-tested independently of SMTP delivery (mock the e-mail service); Mailtrap used only for the end-to-end system test
Four-person team means a single person being unavailable blocks that feature's tests	Each feature owner (Section 9) also documents their test cases in the shared Postman collection so any teammate can run them

11. Assumptions & Dependencies

- The Jenkins CI/CD pipeline, SonarQube and Jira remain available for the duration of the project (SRS Section 2.6).
- Test data (seeded users, the 1,000-task and 500-task performance datasets) will be scripted and available before Sprint 4–5 performance testing begins.
- Each team member can dedicate roughly 14 person-days across the 12-week project (SRS Appendix E), of which testing is embedded per Section 7 above rather than a separate allocation.

12. Suspension & Resumption Criteria

Suspend testing if:

- The Dockerized test environment is unavailable for more than 4 hours.

- The current build is unstable enough to block more than 30% of planned test cases (e.g. the Auth Service is down, blocking every other feature's tests).

Resume testing if:

- The blocking defect(s) are resolved and verified in a new build.
- The test environment is confirmed stable (a smoke-test pass of F1 login succeeds).

13. Test Case Management & Traceability

The Requirement Traceability Matrix below extends the SRS Appendix C matrix with concrete test case IDs (replacing the SRS's "TBD" Verification placeholders for the cases actually designed at this stage) and a representative, executable test case per requirement group. A minimum of one test case is given per feature and per non-functional/security requirement; the full matrix below contains 29 test cases spanning every feature, both performance requirements categories, and all seven security requirements.

TC ID	Requirement(s)	Test Case / Steps	Expected Result	Priority	Type
TC-01	F1-REQ-01, F1-REQ-02	Register with a new e-mail, then attempt to register again with the same e-mail	First registration succeeds (201, session begins); second attempt is rejected with a generic "e-mail already in use" error, no other field is implicated	High	Functional
TC-02	F1-REQ-03, SEC-01	Register a user, then inspect the users table directly	Only a bcrypt hash is stored in password_hash; the plaintext password appears nowhere in the database or logs	High	Security
TC-03	F1-REQ-04, F1-REQ-05	Log in with valid credentials, then with an invalid password	Valid login returns a signed JWT and lands on the dashboard; invalid login returns a generic "invalid e-mail or password" message (401)	High	Functional
TC-04	F1-REQ-08, SEC-04	Call a protected endpoint with an expired JWT	API responds 401 and the client redirects to login	High	Security
TC-05	F1-REQ-09, SEC-02	Call a Project-Owner-only endpoint as a Member-role user	API responds 403; the action is not performed	High	Security
TC-06	F2-REQ-01, F2-REQ-02	Create a task with a valid title, then with an empty title	Valid task is created with status To Do; empty-title request is rejected (400)	High	Functional
TC-07	F2-REQ-04	Move a task from To Do → In Progress → In Review → Done in order, then attempt To Do → Done directly	Sequential transitions succeed and each is logged; the illegal direct jump is rejected	High	Functional
TC-08	F2-REQ-05	Mark a task Done while it has an open (non-Done) blocking dependency	Request is rejected with an explanatory 409 error; task remains in its prior status	High	Functional
TC-09	F2-REQ-06	Declare Task A blocked by Task B,	First declaration succeeds; the	Medium	Functional

		then attempt to declare Task B blocked by Task A	cyclic declaration is rejected		
TC-10	F2-REQ-08	Delete a task that has comments and activity-log entries	Task and its comments/activity-log entries are cascade-deleted	Medium	Functional
TC-11	F3-REQ-03, NFR-01	Search a 1,000-task project by keyword	Matching tasks returned within 1 second (p95), per the defined test load	High	Performance
TC-12	F3-REQ-04, F3-REQ-05	Apply a combined status+priority+assignee filter, then sort by due date	Task list narrows to the intersection of filters and sorts correctly ascending/descending	Medium	Functional
TC-13	F3-REQ-06, F3-REQ-07	Toggle between Kanban and list view, then navigate away and back	Same filtered/sorted task set shown in both views; active filter and view mode preserved on return	Low	Functional
TC-14	F4-REQ-01, F4-REQ-04	Create a project, invite a member, and have the invitee accept	Project created with the creator as Owner; invitee is added with the Member role on acceptance	High	Functional
TC-15	F4-REQ-06	Archive a project with existing tasks	No new tasks can be created in the archived project; existing tasks remain viewable	Medium	Functional
TC-16	F4-REQ-07, SEC-02	Attempt to invite/remove a member as a non-Owner project member	Request rejected with 403	High	Security
TC-17	F5-REQ-01, F5-REQ-02	Assign a task to a project member, then attempt to assign it to a non-member	Valid assignment succeeds and notifies the assignee; invalid assignment is rejected (400)	High	Functional
TC-18	F5-REQ-04	Post a comment that @mentions a project member	A notification is created for each mentioned member	Medium	Functional
TC-19	F5-REQ-07	Attempt to edit or delete an existing comment / activity-log entry (via direct API call)	Request is rejected — comments and activity-log entries are immutable once created	Medium	Functional
TC-20	F6-REQ-02, F6-REQ-03	Run the reminder scheduler twice within the same reminder window for one task	Exactly one reminder e-mail is sent for that task/interval, despite the double run	High	Reliability
TC-21	F6-REQ-04	Let a task reach Done status	No due-date reminder is sent	Medium	Functional

		before its due-date reminder window opens	for that task		
TC-22	F6-REQ-06	Create several notifications for a user and open the notification bell	Unread-notification count matches the number of unread notifications, updates in real time within the session	Low	Functional
TC-23	F7-REQ-02, F7-REQ-04	View a project dashboard with a mix of Done, overdue and in-progress tasks	Completion percentage and overdue count match (Done / total non-archived) and (due date passed, not Done) respectively	Medium	Functional
TC-24	F7-REQ-06, NFR-02	Open the dashboard for a 500-task project	Dashboard renders within 2 seconds under the defined test load	High	Performance
TC-25	F7-REQ-07	Export a project's task list to CSV	Downloaded CSV includes title, assignee, status, priority and due date for every task	Low	Functional
TC-26	F8-REQ-02, F8-REQ-03	Deactivate a user, confirm they cannot log in, then reactivate them	Deactivated user's login is rejected while inactive; historical data is retained; reactivation restores login	High	Functional
TC-27	F8-REQ-06, SEC-06	Trigger a failed login, a role change and an account deactivation	All three events appear in the audit log with actor, event type and timestamp	High	Security
TC-28	SEC-03	Submit a task title containing SQL-metacharacters and an oversized (>200 char) description	Input is sanitized/rejected server-side regardless of client-side validation; no injection occurs	High	Security
TC-29	SEC-07, NFR-03	Submit repeated failed logins against one account; separately, run 20 concurrent users against one project	Repeated failed logins are rate-limited; the system sustains 20 concurrent users while meeting the NFR-01 response-time target	High	Security/ Performance

14. Test Metrics & Reporting

Metrics collected:

- % of the 29 planned test cases executed and % passed/failed, tracked per sprint
- Defect density (defects per feature) and defect aging, tracked in Jira (project TAS)

- Requirement coverage — % of the 70 SRS requirement IDs with at least one passing test case
- Unit test line coverage per service module (SonarQube), against the ≥80% gate

Reports:

- Per-sprint execution status (part of the Sprint 4–6 demo/acceptance evidence in SRS Appendix E)
- Final Test Summary Report, due with Deliverable 4/5

15. Approvals

Role	Name	Signature/Date
QA Lead	Ankita Pramod	
Dev Lead		
Faculty Reviewer		